Software and Tools for Computational Engineering

RWTH AACHEN UNIVERSITY

Sobolev Training and Pruning

. . . or how to make the most out of your AD derivatives

Neil Kichler

Software and Tools for Computational Engineering,[1]

RWTH Aachen University, Aachen, Germany

[1] https://www.stce.rwth-aachen.de

Contents

Motivation

Sobolev Training

Higher-order Sobolev Training

Pruning

Goal

Surrogate models of existing physics models, in the form of:

- Simulation code
- Trained neural network / generative model

Why Surrogates?

- Real-time control
- Inverse Problems
- Sim-to-real finetuning
- Sensitivity analysis

Methods for surrogates

- Vanilla machine learning (ML) on data
- Physics-informed ML (PINNs, Neural xDEs, Neural Operators, etc.)
- Bayesian approaches (MAP, Gaussian Processes, etc.)

However, most of these methods treat the base system code as a *black box*.

Trend:

Differentiable Programming!

Consequence for Surrogates:

Enhanced training without domain specific knowledge.

Metrics

- Accuracy
- Speed of execution

Metrics

- Accuracy
- Speed of execution

Accuracy

Always:

Surrogate must recover input to output mapping.

In many fields (physics):

Surrogate must recover input to derivative/sensitivity mappings!

Metrics

- Accuracy ⇒ Sobolev Training
- Speed of execution

Accuracy

Always:

Surrogate must recover input to output mapping.

In many fields (physics):

Surrogate must recover input to derivative/sensitivity mappings!

⇒ Sobolev Training (i.e., derivative enhanced training)

Metrics

- Accuracy ⇒ Sobolev Training
- Speed of execution

Speed of execution

Always:

Nice to have.

In many scenarios:

Required for feasible *surrogate in the loop* (e.g., control loop).

Metrics

- Accuracy ⇒ Sobolev Training
- Speed of execution ⇒ Pruning

Speed of execution

Always:

Nice to have.

In many scenarios:

Required for feasible *surrogate in the loop* (e.g., control loop).

⇒ Pruning, (Quantization, Distillation, . . .)

Contents

Motivation

Sobolev Training

Higher-order Sobolev Training

Pruning

Goal

Given:

$$f : \mathbb{R}^n \to \mathbb{R}^m, \quad \boldsymbol{y} = f(\boldsymbol{x})$$

Want:

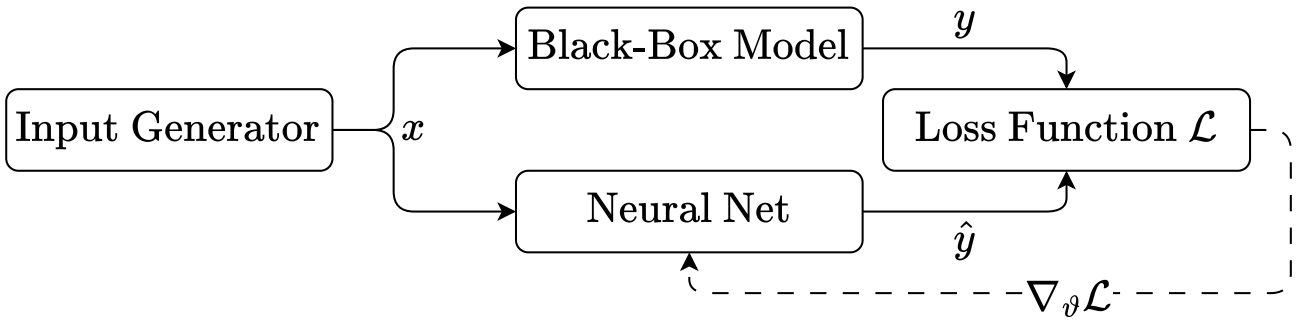
$$\hat{f}_{\boldsymbol{\vartheta}} : \mathbb{R}^n \to \mathbb{R}^m, \quad \hat{\boldsymbol{y}} = \hat{f}_{\boldsymbol{\vartheta}}(\boldsymbol{x}) \approx \boldsymbol{y}$$

with $\boldsymbol{\vartheta}$ fitted using sample data generated by f.

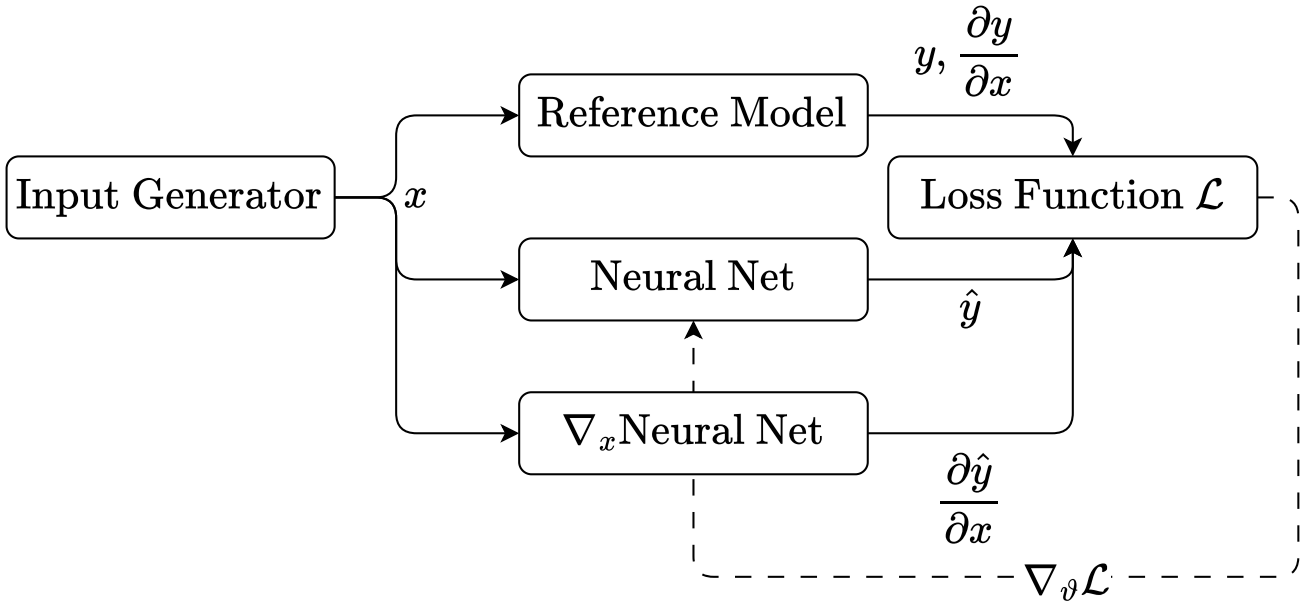
We consider $\hat{f}_{\boldsymbol{\vartheta}}$ to be a neural network.

In the domain of surrogate modeling with neural networks, let:

- $\mathcal{N}(\boldsymbol{\vartheta})$: The surrogate model as a neural network.
  - $\boldsymbol{\vartheta}$: The parameters of the neural network.
  - $f_{\boldsymbol{\vartheta}}$: The learned function.
- $\mathcal{L}$: The loss function (e.g., $\mathcal{L} = \|\cdot\|_2^2$).
- (x_i, y_i): An (input, target) sample.



Match targets by differentiating the loss and optimize using, e.g., SGD.



Match targets and differential targets.

Definition (Sobolev Loss)

Given input $\boldsymbol{x}$, target $\boldsymbol{y}$, predicted output $f_{\boldsymbol{\vartheta}}(\boldsymbol{x})$, derivative target $\nabla_{\boldsymbol{x}}\boldsymbol{y}$, and predicted derivatives $\nabla_{\boldsymbol{x}} f_{\boldsymbol{\vartheta}}(\boldsymbol{x})$, the Sobolev loss is defined by:

$$\|\boldsymbol{y} - f_{\boldsymbol{\vartheta}}(\boldsymbol{x})\|_2^2 + \lambda \|\nabla_{\boldsymbol{x}}\boldsymbol{y} - \nabla_{\boldsymbol{x}} f_{\boldsymbol{\vartheta}}(\boldsymbol{x})\|_2^2,$$

where $\lambda \in \mathbb{R}_{\geq 0}$ is an added balancing factor.

Srinivas and Fleuret [1] highlight that the Sobolev loss naturally arises when considering pertubations.

Perturbation perspective:

Consider perturbation of input. By Taylor expansion [1]:

$$\mathbb{E}_{\boldsymbol{\epsilon}\sim N(0,\sigma^2)}\left[\sum_{i=1}^{m}\left(f(\boldsymbol{x}_i+\boldsymbol{\epsilon})-f_{\boldsymbol{\vartheta}}(\boldsymbol{x}_i+\boldsymbol{\epsilon})\right)^2\right]=$$

$$\sum_{i=1}^{m}\left(f(\boldsymbol{x}_i)-f_{\boldsymbol{\vartheta}}(\boldsymbol{x}_i)\right)^2+\sigma^2\sum_{i=1}^{m}\left\|\nabla_{\boldsymbol{x}}f(\boldsymbol{x}_i)-\nabla_{\boldsymbol{x}}f_{\boldsymbol{\vartheta}}(\boldsymbol{x}_i)\right\|_2^2+\mathcal{O}(\sigma^4).$$

Algorithm Sobolev Training [2].

Require: The following inputs must all be initialized.

- Surrogate model $\mathcal{N}(\boldsymbol{\vartheta})$ with function $f_{\boldsymbol{\vartheta}}$ and parameters $\boldsymbol{\vartheta}$
- Reference model $\mathcal{S}$
- Optimizer G

while $\boldsymbol{\vartheta}$ not converged **do**

$\{(\boldsymbol{x}_i, \boldsymbol{y}_i, \nabla_{\boldsymbol{x}} \boldsymbol{y}_i)\}_{i=1}^{m} \sim \mathcal{S}$ ▷ Sample training data

$\hat{\boldsymbol{g}} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\vartheta}} \sum_{i=1}^{m} \mathcal{L}(f_{\boldsymbol{\vartheta}}(\boldsymbol{x}_i), \boldsymbol{y}_i) + \lambda \mathcal{L}(\nabla_{\boldsymbol{x}} f_{\boldsymbol{\vartheta}}(\boldsymbol{x}_i), \nabla_{\boldsymbol{x}} \boldsymbol{y}_i)$

$\boldsymbol{\vartheta} \leftarrow G(\boldsymbol{\vartheta}, \hat{\boldsymbol{g}})$ ▷ Update parameters

end while

return $\mathcal{N}$

Consider the SDE:

$$\mathrm{d}S_t = a(S,t)\,\mathrm{d}t + b(S,t)\,\mathrm{d}W_t,$$

where:

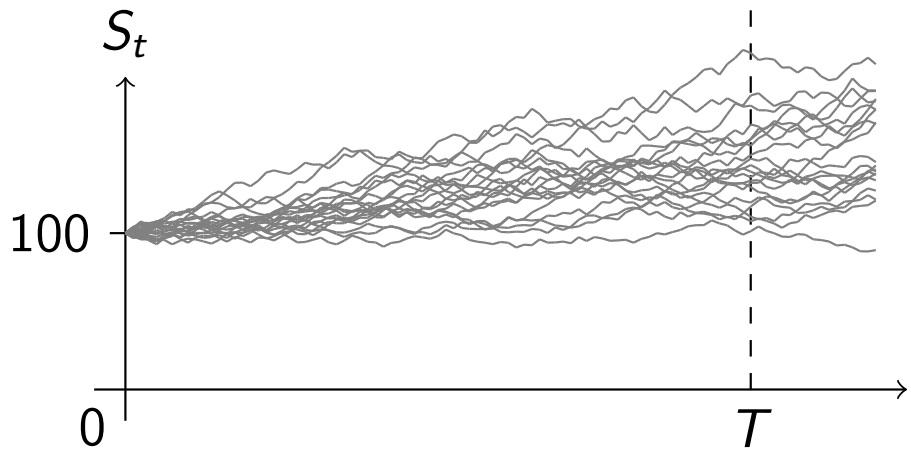
- $\mathrm{d}W_t$ denotes a Wiener process
- $a : \mathbb{R} \times [0,T] \to \mathbb{R}$ is the drift
- $b : \mathbb{R} \times [0,T] \to \mathbb{R}$ is the volatility

Interested in the price:

$$V = \mathbb{E}[\nu(S_T, K)],$$

with:

- S_T the price at maturity T
- K the strike price
- $\nu(S_T, K)$ the payoff function



Sample paths of SDE.

The Euler-Maryuama discretization for the SDE is given by:

$$\hat{S}_{i+1} = \hat{S}_i + a(\hat{S}_i, t_i)\Delta t + b(\hat{S}_i, t_i)\Delta W_i,$$

where

- $t \in [0, T]$
- $\Delta W_i \sim \sqrt{\Delta t} N(0, 1)$
- $\Delta t = \frac{T}{n}$
- $i \in \mathbb{N}$, $0 \leq i < n$
- n time steps

MC estimator is unbiased:

$$\mathbb{E}[\hat{V}] = \mathbb{E}\Big[\frac{1}{m}\sum_{i=1}^{m} \nu(\hat{S}_T^{(i)}, K)\Big] = \frac{1}{m}\sum_{i=1}^{m} \mathbb{E}[\nu(\hat{S}_T^{(i)}, K)] = \mathbb{E}[\nu(S_T, K)] = V.$$

We are, furthermore, interested in the *Greeks* (parameter sensitivities):

$$\frac{\partial V}{\partial \boldsymbol{\theta}},$$

where $\boldsymbol{\theta}$ represents an input parameter. E.g., $\boldsymbol{\theta} = S_0$ for the Delta.

How do we efficiently compute the Greeks?
$\Rightarrow$ adjoint AD.

But:

Requires a lot of memory: must store all intermediate time steps for all paths.

Better:

Can we find the Greeks by computing the derivative for each path separately?
$\Rightarrow$ *Yes*! With pathwise derivatives [3].

Fix some random sample path $z \sim \mathcal{Z}$ and input parameters $\boldsymbol{\theta}$.

We have unbiased estimates of, e.g., pathwise deltas, if:

$$\mathbb{E}_{z\sim\mathcal{Z}}\left[\frac{\partial}{\partial S_0}\nu(g(\boldsymbol{\theta}, z))\right] = \frac{\partial}{\partial S_0}\mathbb{E}_{z\sim\mathcal{Z}}\left[\nu(g(\boldsymbol{\theta}, z))\right], \tag{1}$$

i.e. we can interchange the expectation with the derivative operator [4].

Practical Conditions for (1):

- Payoff ν must be differentiable almost everywhere.
- Payoff ν is Lipschitz continuous.

Example (European Call Option):

$$\frac{\partial \nu(g(\boldsymbol{\theta}, z))}{\partial F_T} = \frac{\partial}{\partial F_T}(F_T - K)^+ = \mathbb{1}_{F_T > K}.$$

Note:

- Payoff: $\nu = (F_T - K)^+ = \max(F_T - K, 0)$.
- At $F_T = K$ the derivative does not exist.
- Event $F_T = K$ occurs with probability 0.
- Payoff function is a.s. differentiable with respect to F_T.
- It is Lipsichitz continuous:

$$\|\nu(y_1) - \nu(y_2)\| = \|(y_1)^+ - (y_2)^+\| = \left\{ \begin{array}{ll} \|y_1 - y_2\|, & \text{if } y_1 > 0, y_2 > 0 \\ \|y_1\|, & \text{if } y_1 > 0, y_2 < 0 \\ 0, & \text{if } y_1 < 0, y_2 < 0 \end{array} \right\} \leq \|y_1 - y_2\|.$$

Consider the stochastic volatility model:

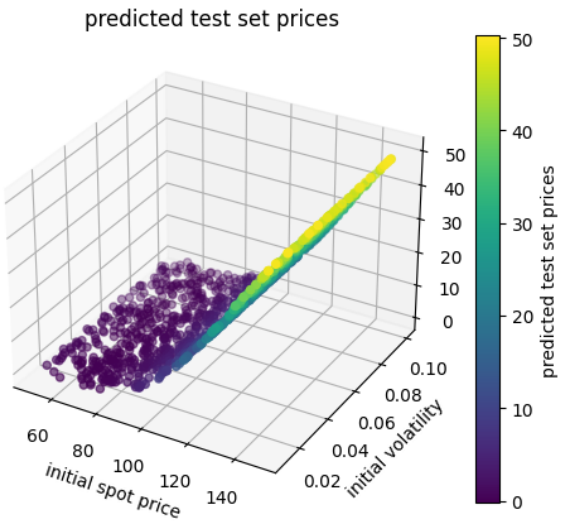
$$\begin{aligned} \mathrm{d}S_t &= \mu S_t \,\mathrm{d}t + \sqrt{\nu_t} S_t \,\mathrm{d}W_t^{\,S}, \\ \mathrm{d}\nu_t &= \kappa(\theta - \nu_t)\,\mathrm{d}t + \xi\sqrt{\nu_t}\,\mathrm{d}W_t^{\,\nu}, \end{aligned} \tag{2}$$

where:

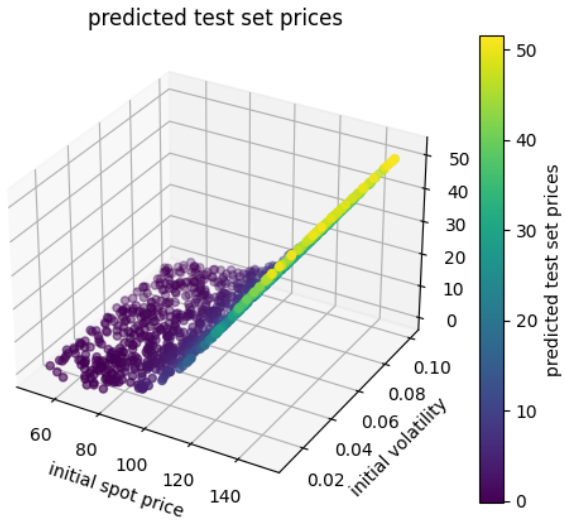
- μ is the drift of the underlying asset,
- ν the instantaneous variance,
- θ the long run average variance of the price,
- κ the rate of mean reversion of ν_t to θ
- ξ the volatility of the volatility.
- W_t^S is correlated to W_t^ν with ρ, i.e. $\mathbb{E}[\mathrm{d}W_t^{\,S}\,\mathrm{d}W_t^{\,\nu}] = \rho\,\mathrm{d}t$.

We apply it to a European call option (i.e., payoff $\mathrm{v} = (\cdot)^+$).

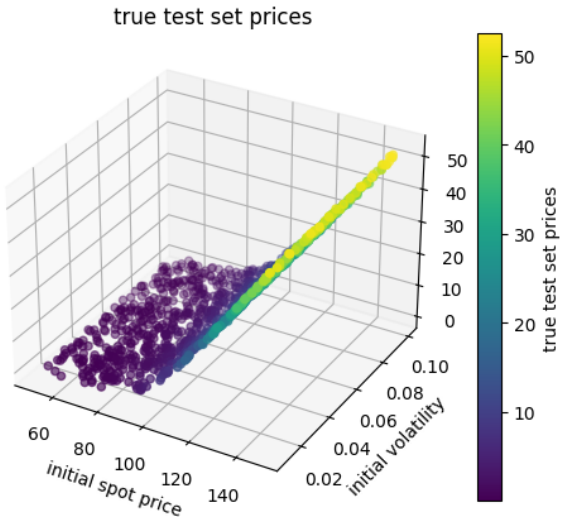
Comparison of RMSE for predicted prices V:



(a) Vanilla ML



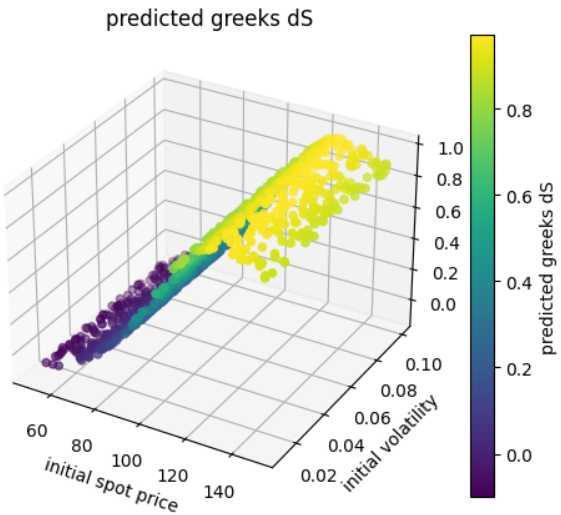
(b) Differential ML



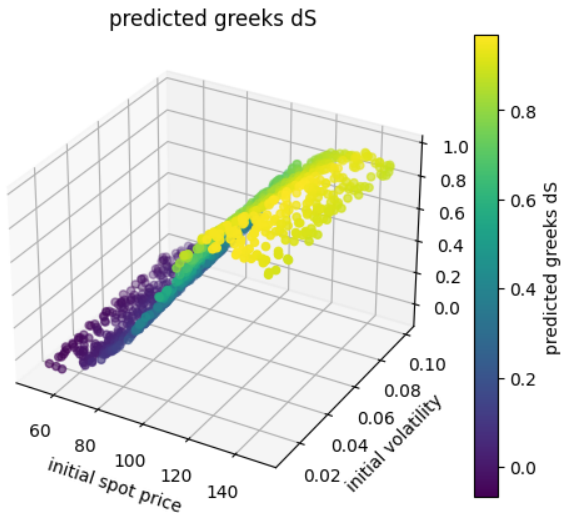
(c) Test Set

RMSE: $0.0557 \rightarrow 0.0157$

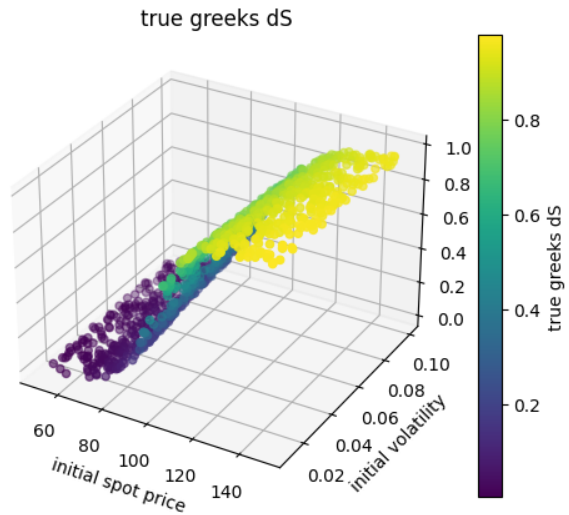
Comparison of RMSE for predicted Deltas $\frac{\partial V}{\partial S_0}$:



(a) Vanilla ML



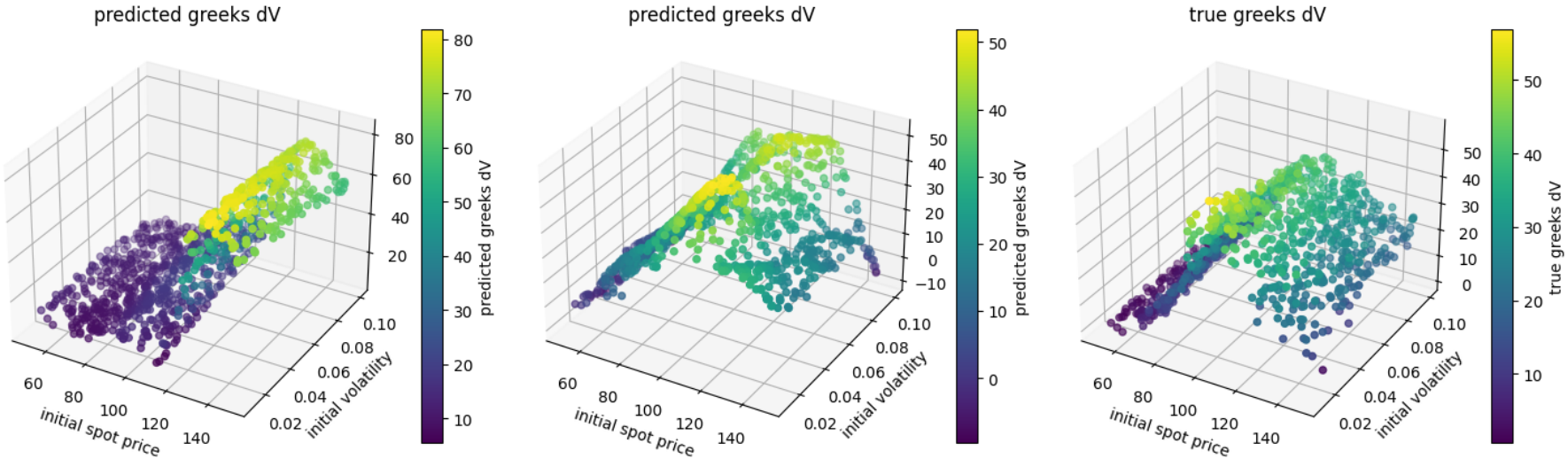
(b) Differential ML



(c) Test Set

RMSE: 0.0602 $\rightarrow$ 0.0361

Comparison of RMSE for predicted Vegas $\frac{\partial V}{\partial \sigma}$:



(a) Vanilla ML (b) Differential ML (c) Test Set

RMSE: 0.0281 $\rightarrow$ 0.0073

Similar Ideas exist,

in physics informed methods:

- gPINNs [5]
- DINO (derivative informed neural operators) [6]

in control using reinforcement learning:

- GPPO (gradient-enhanced PPO)
- see HydroGym and control of flow ([7])

Contents

Motivation

Sobolev Training

Higher-order Sobolev Training

Pruning

Can we extend this to higher-order derivative information?

Can we extend this to higher-order derivative information? Yes!

But, challenges arise:

- non-differentiability
- computational complexity
- ...

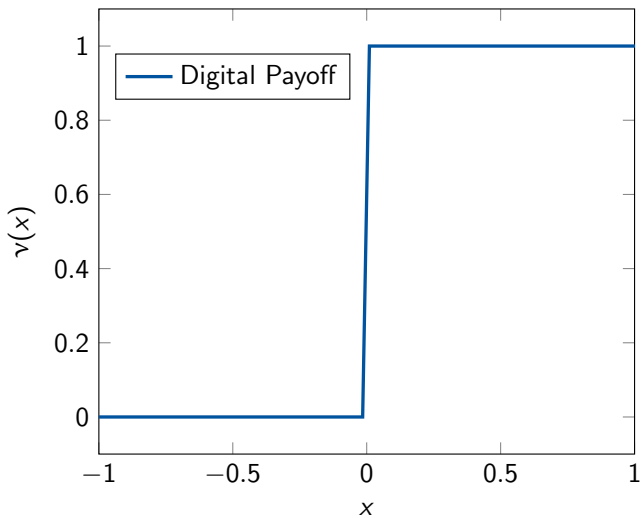
Remember, practical conditions for pathwise AD:

- Payoff ν must be differentiable almost everywhere.
- Payoff ν is Lipschitz continuous.

What if they are not fulfilled?
(e.g., Heaviside function)

Digital Payoff:

- Payoff is differentiable almost everywhere, except at $S_T = K$.
- But: all values are 0!
  $\hookrightarrow$ Dirac Delta function is not modelled correctly through pathwise derivatives.
- Not Lipsichitz continuous because Heaviside is not even continuous.



$\Rightarrow$ need for smoothing arises.

We perform smoothing between function $f_1 : \mathbb{R} \to \mathbb{R}$ and $f_2 : \mathbb{R} \to \mathbb{R}$ via $\tilde{f} : \mathbb{R} \times \mathbb{R}^2 \to \mathbb{R}$ defined as

$$\tilde{f}(x, p, w) = (1 - \sigma(x, p, w))f_1(x) + \sigma(x, p, w)f_2(x),$$

where

$$\sigma(x, p, w) = \frac{1}{1 + e^{-\frac{x-p}{w}}},$$

and p is the position to change between the two functions and w the width of the smoothing.

Sigmoidal Smoothing

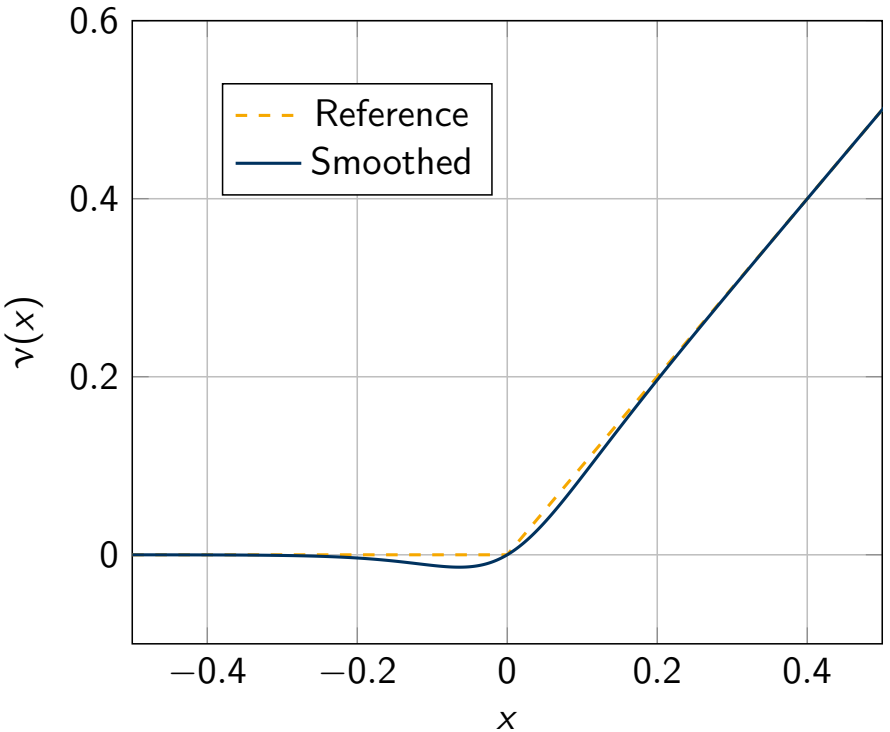
For $\nu = (\cdot)^+$:

Split function into $\begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}$.

We obtain:

$$\tilde{\nu}(x, w) = \frac{x}{1 + e^{-\frac{x}{w}}}.$$

Equivalent to SiLU if $p = 0, w = 1$.

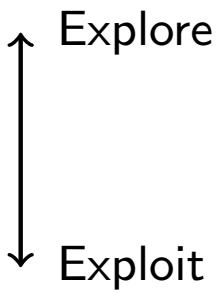


Smoothing ν with $p = 0, w = 0.05$.

Second-order sensitivity information

Hessian too expensive? $\rightarrow$ sample directions:

- Random directions
- Randomized LA (i.e., sketching)
- PCA directions (of derivatives)



Hutchinson's Estimator

Draw random vectors $\boldsymbol{v}$, s.t. $\mathbb{E}[\boldsymbol{v}\boldsymbol{v}^T] = \boldsymbol{I}$.

$$\Rightarrow \mathbb{E}\left[\boldsymbol{H}\boldsymbol{v}\boldsymbol{v}^T\right] = \boldsymbol{H}\mathbb{E}\left[\boldsymbol{v}\boldsymbol{v}^T\right] = \boldsymbol{H}.$$

E.g., $\boldsymbol{v} \sim N(\boldsymbol{\mu} = \mathbf{0}, \boldsymbol{\Sigma} = \boldsymbol{I})$.

Note:

- Apply outer product after computation of the HVP (i.e., $(\boldsymbol{H}\boldsymbol{v})\boldsymbol{v}^\mathsf{T}$).
- Other distributions possible (e.g., Rademacher).

Taking inspiration from (randomized) Linear Algebra:

Sketching with randomized HVP probes [8] (e.g., randomized SVD)

- Let $Y = H\Omega, \quad \Omega \sim N(0, I)$
- $Q = \text{orth}(Y)$ $(Y = QR)$
- $B = Q^\mathsf{T} H Q$
- work with small matrix B $(B = \tilde{U}\Sigma V^\mathsf{T})$
- map back to original space with Q. $(U = Q\tilde{U})$

Sample HVPs from (randomized, incremental) PCA [9]

- Compute principal components.
- Take k-most significant components.
- Directional derivatives in direction of principal components.

Consider the Bachelier model for a call option with European payoff:

$$\mathrm{d}F_t = \sigma \, \mathrm{d}W_t, \tag{3}$$

where:

- $\mathrm{d}W_t$ denotes a Wiener process
- σ is the constant volatility

This is a simplified view using a forward measure and forward price:

$$F_t = S_t e^{r(T-t)},$$

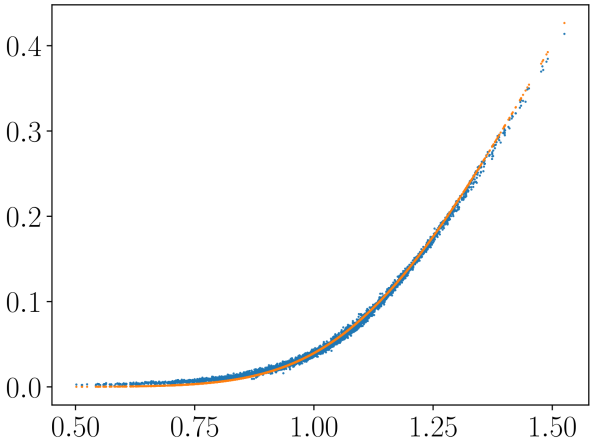
where r is the risk free interest rate.

Furthermore, consider Gaussian basket: $\mathbf{F}_t = \sum_{i=0}^{m} \omega_i F_t^{[i]}$.

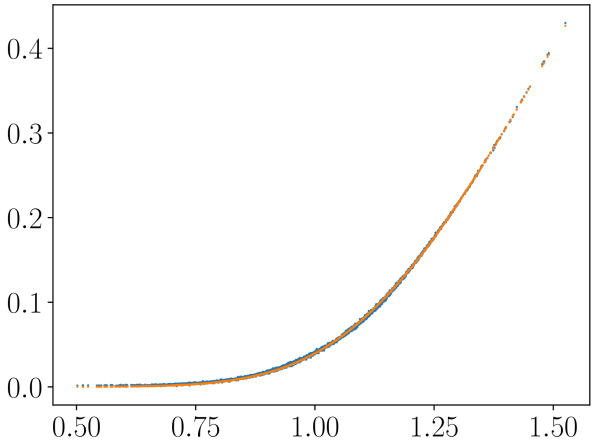
- Weighted sum of underlying jointly Gaussian assets
- Basket price $\mathbf{F}_t$ is again Gaussian.

Results: Bachelier Basket

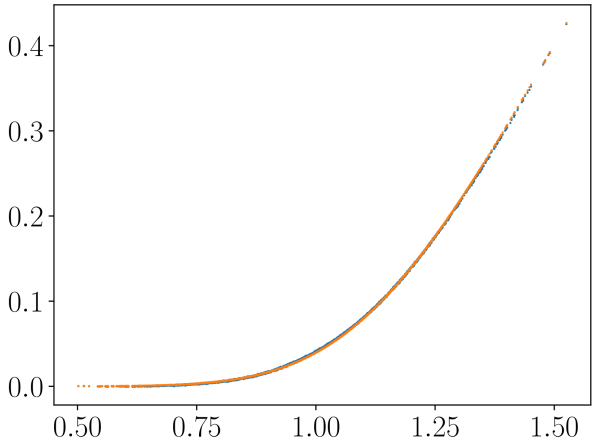
Comparison of RMSE of predicted prices:



Vanilla ML



Differential ML



Second-Order DML

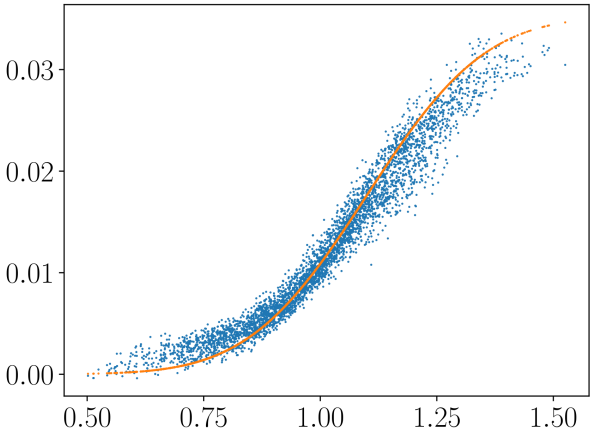
RMSE: $0.35 \rightarrow 0.13 \rightarrow 0.08$

30 runs:

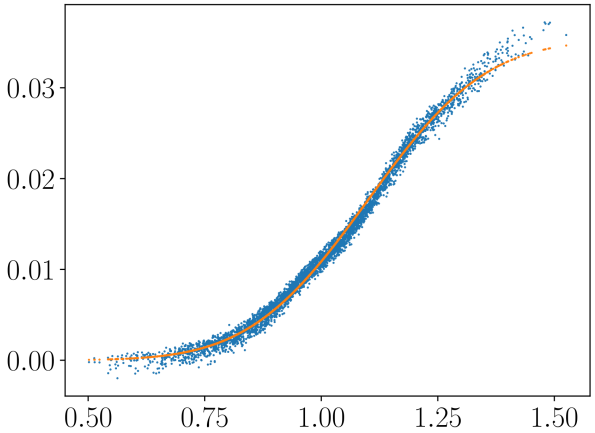
# Dim	# Samples	Standard ML	Differential ML	2nd-Order PCA
7	8192	0.320 ± 0.022	0.123 ± 0.009	0.101 ± 0.016

Results: Bachelier Basket

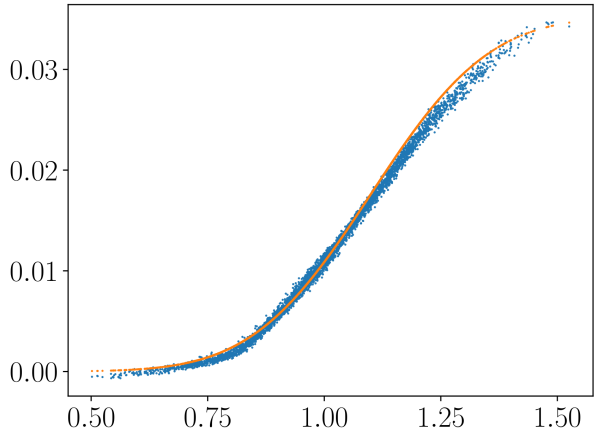
Comparison of RMSE of predicted Deltas:



Vanilla ML



Differential ML



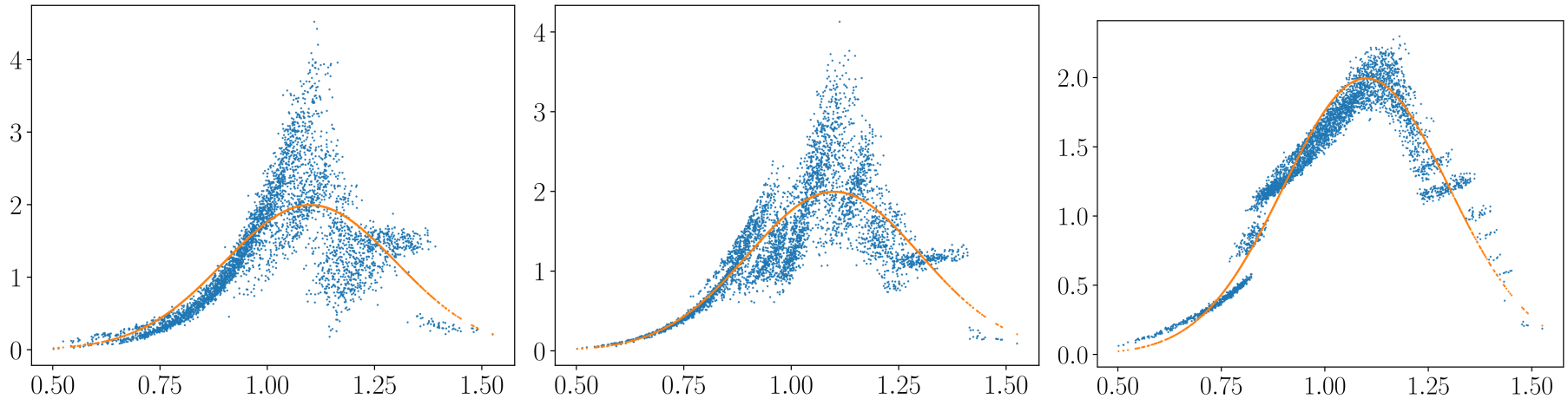
Second-Order DML

RMSE: $0.53 \rightarrow 0.26 \rightarrow 0.13$

30 runs:

# Dim	# Samples	Standard ML	Differential ML	2nd-Order PCA
7	8192	0.532 ± 0.009	0.261 ± 0.025	0.155 ± 0.018

Comparison of RMSE of predicted Gammas:

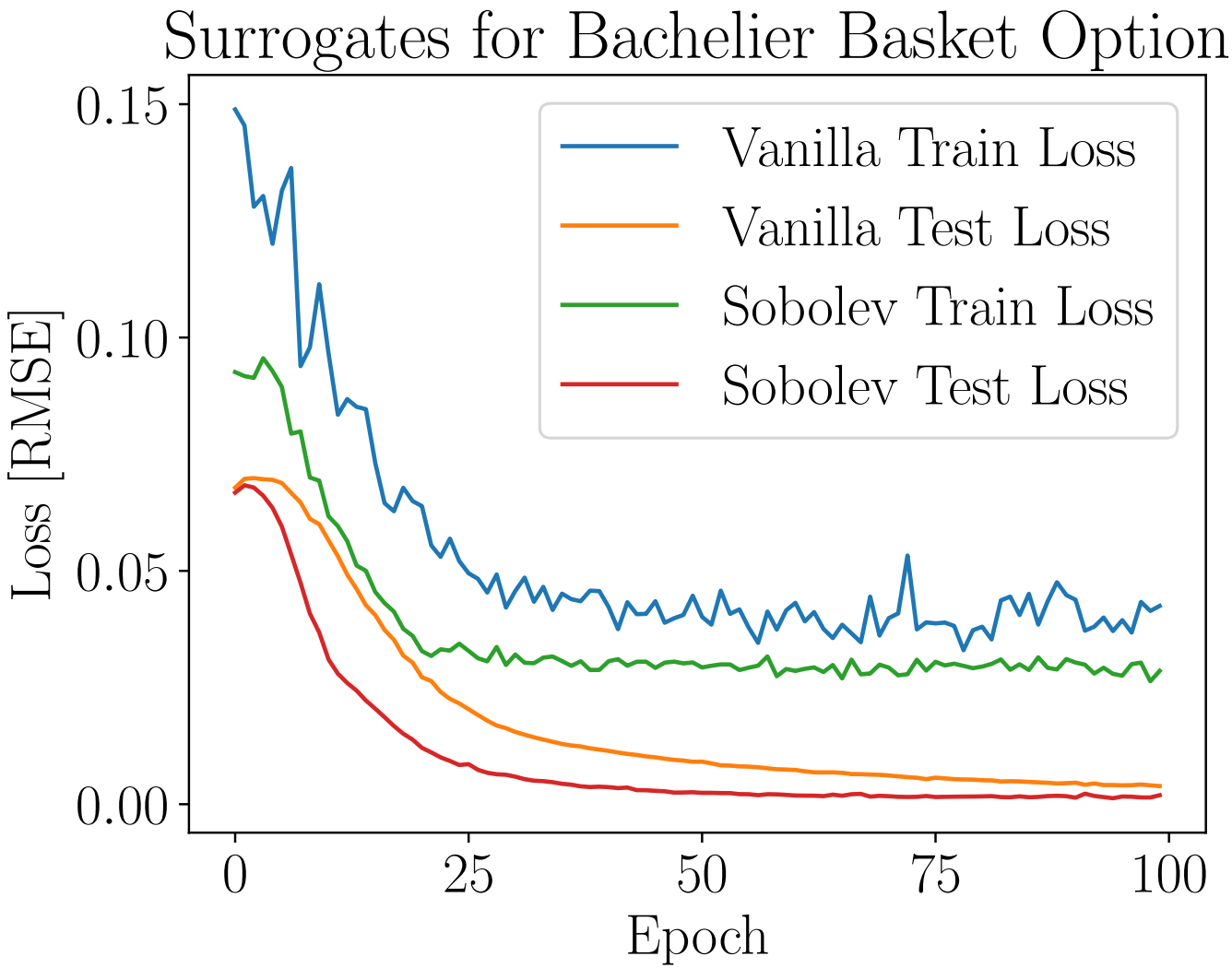


(a) Vanilla ML (b) Differential ML (c) Second-Order DML

RMSE: $97.5 \rightarrow 87.3 \rightarrow 74.7$

30 runs:

# Dim	# Samples	Standard ML	Differential ML	2nd-Order PCA
7	8192	97.625 ± 0.334	87.390 ± 2.201	75.54 ± 0.506


Surrogates for Bachelier Basket Option
Vanilla Train Loss
Vanilla Test Loss
Sobolev Train Loss
Sobolev Test Loss
Loss [RMSE]
0.15
0.10
0.05
0.00
0
25
50
75
100
Epoch

Contents

Motivation

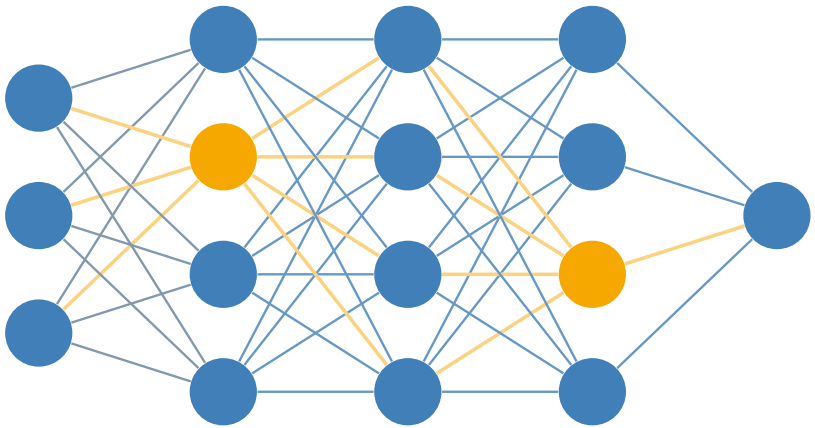
Sobolev Training

Higher-order Sobolev Training

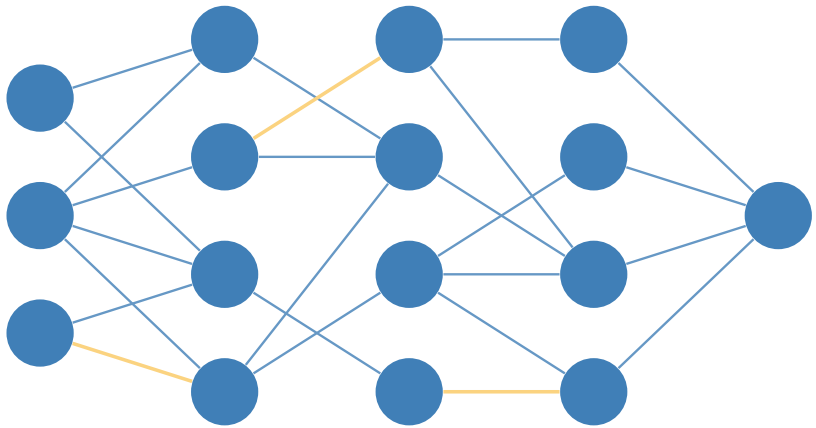
Pruning

Goal

Prune the surrogate model to increase the computational efficiency while retaining accuracy.



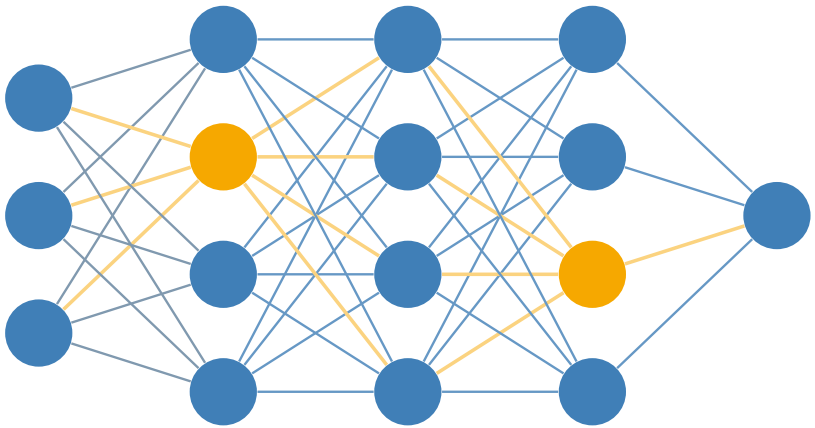
Dense Pruning.



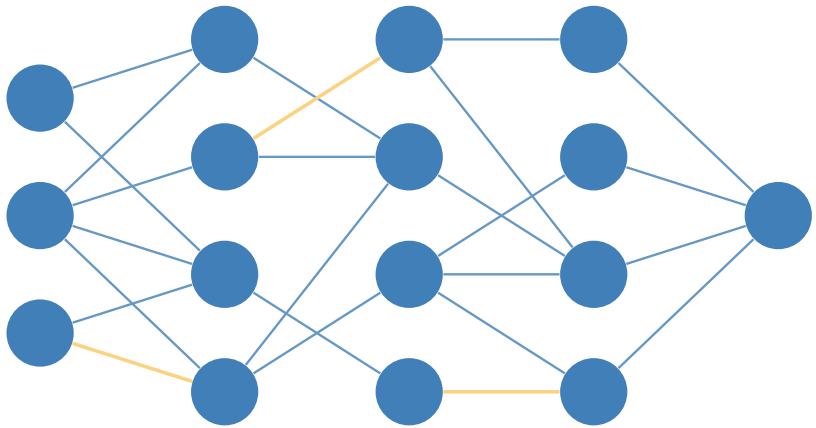
Sparse Pruning.

Goal

Prune the surrogate model to increase the computational efficiency while retaining accuracy.



Dense Pruning.



Sparse Pruning.

In practice: Many neural networks can be pruned by $> 80\%$ with insignificant loss of accuracy.

Typically

Magnitude Pruning: $|w|$

Salience Pruning: $|\frac{\partial \mathcal{L}}{\partial w} w|$

...

Typically

Magnitude Pruning: $|w|$

Salience Pruning: $|\frac{\partial \mathcal{L}}{\partial w} w|$

...

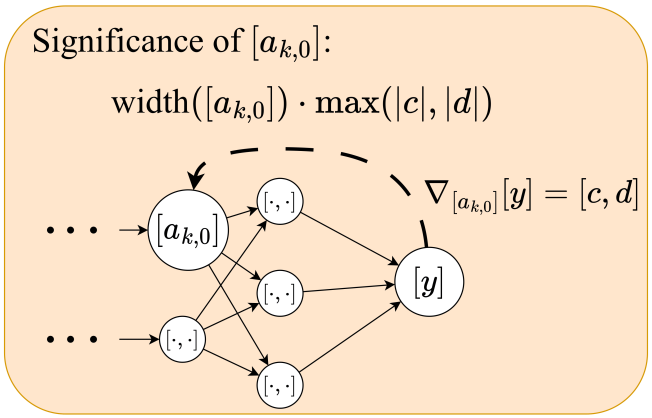
Downsides?

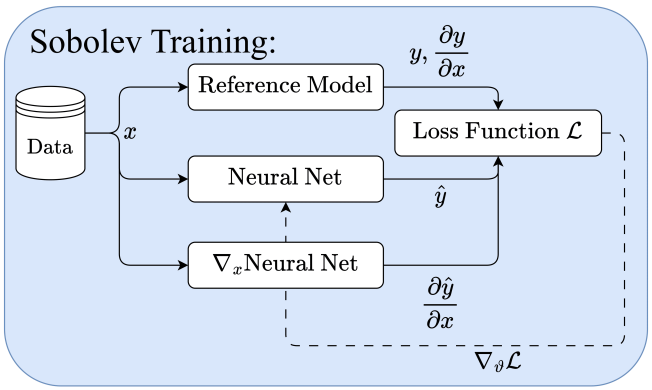
- Must iterate over training data.
- Data sensitivity information?
- No global perspective.

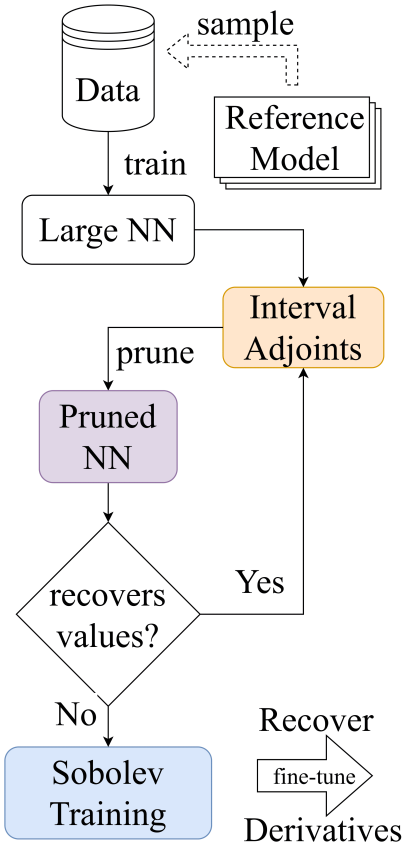
Idea:

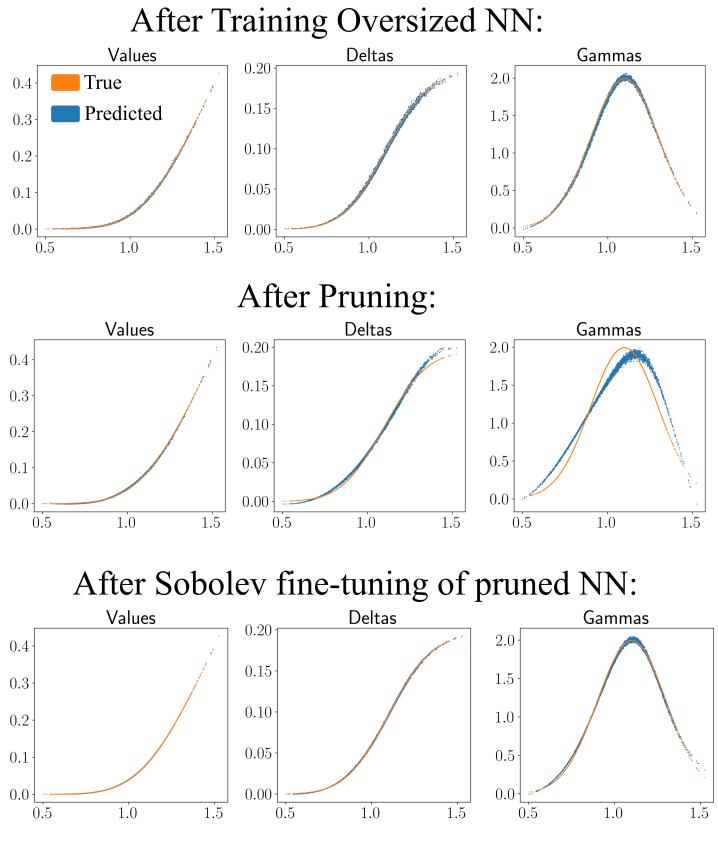
Use AD + Interval Arithmetic + Sobolev Training

Pruning









Differential data can be used for model pruning and model distillation.

More info: github.com/neilkichler/sobolev-pruning and PASC '24, [10].

Differentiable Programming:

- A foundation to build on for modern simulations.

Differentiable Programming:

- A foundation to build on for modern simulations.

Sobolev Training:

- Derivatives can aid training & final accuracy.
- Pathwise adjoints for efficient derivatives of Monte Carlo simulations.
- Watch out: Differentiable? Jacobian/Hessian comp. feasible?

Differentiable Programming:

- A foundation to build on for modern simulations.

Sobolev Training:

- Derivatives can aid training & final accuracy.
- Pathwise adjoints for efficient derivatives of Monte Carlo simulations.
- Watch out: Differentiable? Jacobian/Hessian comp. feasible?

Pruning:

- Prune from large to small neural network.
- Easy $\sim 80\%$ size reduction with minor performance degradation.
- Recover derivative information with finetuning using Sobolev Training.

Differentiable Programming:

- A foundation to build on for modern simulations.

Sobolev Training:

- Derivatives can aid training & final accuracy.
- Pathwise adjoints for efficient derivatives of Monte Carlo simulations.
- Watch out: Differentiable? Jacobian/Hessian comp. feasible?

Pruning:

- Prune from large to small neural network.
- Easy $\sim 80\%$ size reduction with minor performance degradation.
- Recover derivative information with finetuning using Sobolev Training.

That's it! Any questions?

[1] Suraj Srinivas and Francois Fleuret. "Knowledge Transfer with Jacobian Matching". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. PMLR, July 2018, pp. 4723–4731. arXiv: 1803.00443.

[2] Wojciech M Czarnecki et al. "Sobolev Training for Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017. arXiv: 1706.04859.

[3] Mike Giles and Paul Glasserman. "Smoking adjoints: Fast Monte Carlo Greeks". In: *Risk* 19.1 (2006), pp. 88–92.

[4] Paul Glasserman. *Monte Carlo Methods in Financial Engineering*. Springer New York, NY, 2003. DOI: 10.1007/978-0-387-21617-1.

[5] Jeremy Yu et al. "Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems". In: *Computer Methods in Applied Mechanics and Engineering* 393 (2022), p. 114823. DOI: https://doi.org/10.1016/j.cma.2022.114823.

[6] Thomas O'Leary-Roseberry et al. "Derivative-Informed Neural Operator: An efficient framework for high-dimensional parametric derivative learning". In: *Journal of Computational Physics* 496 (2024), p. 112555. DOI: https://doi.org/10.1016/j.jcp.2023.112555.

[7] Christian Lagemann et al. "Hydrogym: A reinforcement learning platform for fluid dynamics". In: *arXiv preprint arXiv:2512.17534* (2025).

[8] Johannes Wilhelm. "Efficient Higher-Order Sobolev Training via Randomized Hessian Sketching". Master's thesis. Software and Tools for Computational Engineering Group, RWTH Aachen University, 2025.

[9] Neil Kichler. "Second-Order Differential Machine Learning". Master's thesis. Software and Tools for Computational Engineering Group, RWTH Aachen University, 2023.

[10] Neil Kichler et al. "Towards Sobolev Pruning". In: *Proceedings of the Platform for Advanced Scientific Computing Conference*. PASC '24. Zurich, Switzerland: Association for Computing Machinery, 2024. DOI: 10.1145/3659914.3659915.

Backup Slides

We normalize the data samples for each dimension:

$$\tilde{\boldsymbol{x}}_i = \frac{\boldsymbol{x}_i - \boldsymbol{\mu}_x}{\boldsymbol{\sigma}_x}, \quad \tilde{\boldsymbol{y}}_i = \frac{\boldsymbol{y}_i - \boldsymbol{\mu}_y}{\boldsymbol{\sigma}_x}.$$

To normalize the differential targets, consider some element x of $\boldsymbol{x}_i$ and y of $\boldsymbol{y}_i$. Then, by the chain rule

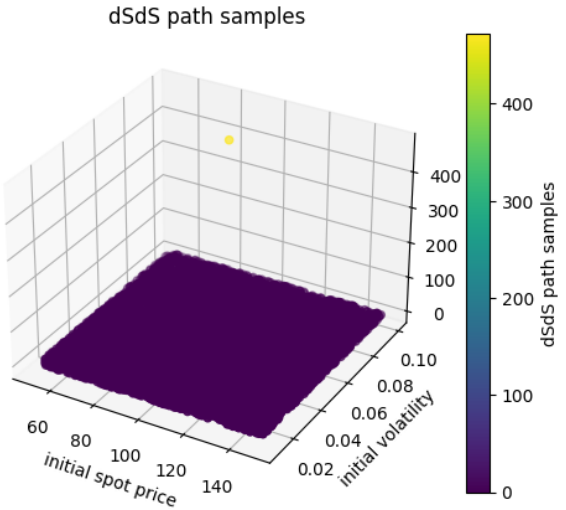
$$\frac{\partial \tilde{y}}{\partial \tilde{x}} = \frac{\partial \tilde{y}}{\partial y}\frac{\partial y}{\partial x}\frac{\partial x}{\partial \tilde{x}} = \frac{\sigma_x}{\sigma_y}\frac{\partial y}{\partial x},$$

where by linearity of the derivative:

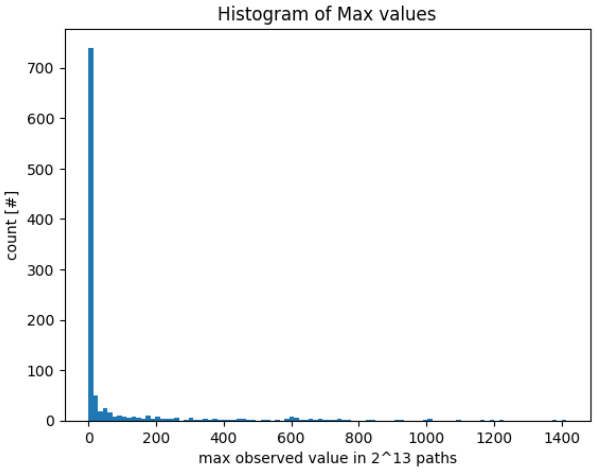
$$\frac{\partial \tilde{y}}{\partial y} = \frac{\partial}{\partial y}\frac{(y - \mu_y)}{\sigma_y} = \frac{1}{\sigma_y},$$

$$\frac{\partial x}{\partial \tilde{x}} = \frac{\partial}{\partial \tilde{x}}(\sigma_x \tilde{x} + \mu_x) = \sigma_x.$$

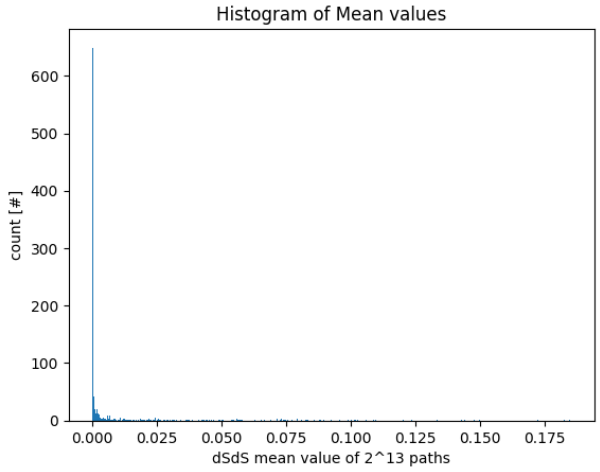
Why did we not show second-order results?
↪ Variance and kurtosis are huge with Euler-Maryuama Scheme



(a) Training batch example



(b) Max per batch

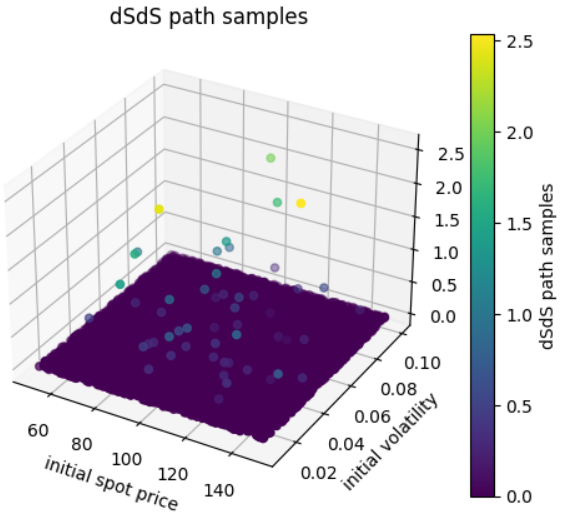


(c) Mean per batch

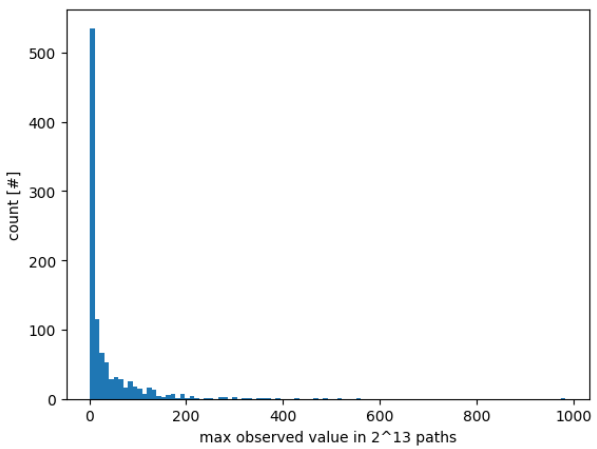
Requires much better variance reduction:

- Antithetic paths and better discretization schemes
- Vibrato Monte Carlo

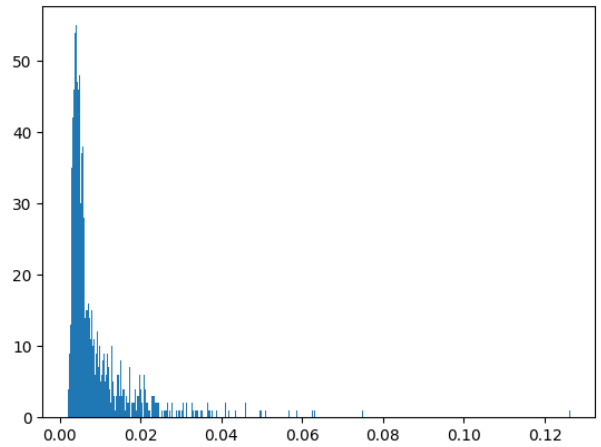
Vibrato MC considers distribution over final time step.



(a) Training batch example



(b) Max per batch



(c) Mean per batch

- Promising. . . but implementation is tricky.
- Simpler variance reduction might be sufficient.